LCA, Binary Lifting & Euler Tours

Answer "who is the common boss?" in O(log n), and turn subtrees into array ranges

Learning goals

- Define the **lowest common ancestor** (LCA) and use it for tree distances
- Build a **binary lifting** table `up[j][v]` = 2^j-th ancestor of `v`
- Answer **k-th ancestor** and **LCA** queries in O(log n)
- Flatten a tree with an **Euler tour** (`tin` / `tout`)
- Reduce **subtree queries** and **path-to-root queries** to array range queries
- Know the alternative: LCA as **RMQ** on the Euler tour

Where we are

- Week 16: BFS / DFS on graphs
- Week 20: trees — depth, subtree sizes, tree DP
- Week 21: DAGs, topological order
- **Today:** fast queries on *static* tree structure
- Next weeks: segment trees (23), Fenwick & sparse tables (24) — the data structures we will plug into the flattened tree

Motivating problem — CSES 1135 *Distance Queries*

- Tree with n ≤ 2·10^5 nodes, q ≤ 2·10^5 queries
- Each query: distance (number of edges) between `a` and `b`
- BFS per query: O(n·q) = 4·10^10 — far too slow

```
    1              dist(4, 2)?
   / \             path 4 → 3 → 1 → 2  = 3 edges
  2   3
     / \          key node on the path: 1 = LCA(4, 2)
    4   5
```

Lowest common ancestor

- Root the tree (say at node 1). `depth[root] = 0`
- `u` is an **ancestor** of `v` if `u` lies on the path from `v` to the root (a node is its own ancestor)
- **LCA(a, b)** = the deepest node that is an ancestor of both

Distance formula

```
dist(a, b) = depth[a] + depth[b] - 2 * depth[LCA(a, b)]
```

The path a → b goes up to the LCA and then back down.

Naive LCA

```
int lcaNaive(int a, int b) {
    while (depth[a] > depth[b]) a = par[a];
    while (depth[b] > depth[a]) b = par[b];
    while (a != b) { a = par[a]; b = par[b]; }
    return a;
}
```

- Correct, but O(depth) = O(n) per query on a path-like tree
- Idea: take **big jumps** instead of single steps

Binary lifting: the table

`up[j][v]` = ancestor of `v` that is **2^j** levels above (clamp at the root)

```
up[0][v] = parent(v)
up[j][v] = up[j-1][ up[j-1][v] ]      // 2^j = 2^(j-1) + 2^(j-1)
```

- LOG = ⌈log2 n⌉ + 1 levels → n · LOG integers (≈ 3.6M for n = 2·10^5)
- Build in O(n log n)
- Requirement: fill **level by level** (whole level j-1 before level j), or in BFS/DFS order

Example table

```
tree: 1 - 2 - 3 - 4 - 5 - 6 (a path, rooted at 1)

 v       :  1  2  3  4  5  6
 up[0][v]:  1  1  2  3  4  5
 up[1][v]:  1  1  1  2  3  4
 up[2][v]:  1  1  1  1  1  2
```

`up[2][6]` = 4 levels above 6 = node 2 ✔

k-th ancestor in O(log n)

Write k in binary; jump by each set bit.

```
int kth(int v, int k) {
    for (int j = 0; j < LOG; j++)
        if (k >> j & 1) v = up[j][v];
    return v;
}
```

- k = 13 = 8 + 4 + 1 → three jumps
- Order of jumps does not matter (they commute)
- Use a sentinel 0 ("no node") instead of clamping if you must report −1 (CSES 1687)

LCA in O(log n): two phases

1. **Equalize depths**: lift the deeper node by `depth[a] - depth[b]`
2. If equal now → that node is the LCA
3. **Lift both together** from the largest jump down: jump only if the targets **differ**
4. Answer = parent of the final `a`

Invariant of phase 3: `a ≠ b` but they always have the same depth; we never jump past the LCA.

LCA code

```
int lca(int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = kth(a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int j = LOG - 1; j >= 0; j--)
        if (up[j][a] != up[j][b]) {
            a = up[j][a];
            b = up[j][b];
        }
    return up[0][a];
}
```

Why "differ"? If `up[j][a] == up[j][b]` the jump lands **on or above** the LCA — we cannot tell which, so we refuse it. This is binary search on the depth of the LCA.

Trace: LCA(4, 2)

```
      1          depth: 1→0, 2→1, 3→1, 4→2, 5→2
     / \
    2   3        phase 1: depth[4]=2 > depth[2]=1, lift 4 by 1 → 3
       / \       3 ≠ 2
      4   5      phase 3 (LOG = 3): j = 2, 1, 0 — every jump from 3
                 and from 2 lands on 1 → equal → skip
                 answer: up[0][3] = 1
```

dist(4, 2) = 2 + 1 − 2·0 = **3** ✔

Building without recursion

Recursive DFS on a 2·10^5-node path can overflow the stack. Use BFS order:

```
vector<int> order = {1};
up[0][1] = 1; depth[1] = 0; seen[1] = true;
for (size_t i = 0; i < order.size(); i++) {
    int v = order[i];
    for (int u : adj[v]) if (!seen[u]) {
        seen[u] = true; up[0][u] = v;
        depth[u] = depth[v] + 1; order.push_back(u);
    }
}
for (int j = 1; j < LOG; j++)
    for (int v = 1; v <= n; v++)
        up[j][v] = up[j-1][up[j-1][v]];
```

Binary lifting carries extra data

Store aggregates along jumps, e.g. `mx[j][v]` = max edge weight on the 2^j edges above `v`:

```
mx[j][v] = max(mx[j-1][v], mx[j-1][ up[j-1][v] ])
```

- Max edge on a path a–b: combine values while lifting both sides to the LCA
- Classic use: **CF 609E** — for each edge, the MST that contains it (Week 19 + today)
- Works for any **associative** operation (sum, min, gcd, ...)

Euler tour: flatten the tree

DFS and record the time we **enter** each node.

```
      1          visit order: 1 2 3 4 5
     / \         tin : 1→1, 2→2, 3→3, 4→4, 5→5
    2   3        tout: 1→5, 2→2, 3→5, 4→4, 5→5
       / \       (tout = last tin inside the subtree)
      4   5
```

Key fact: subtree of `v` = positions `[tin[v], tout[v]]` — a **contiguous range**.

Ancestor test in O(1)

```
u is an ancestor of v  ⇔  tin[u] ≤ tin[v] and tout[v] ≤ tout[u]
```

- Lets you write LCA with a single lifting loop (no depth array needed)
- Also handy on its own: "is `v` in the subtree of `u`?"

Subtree queries — CSES 1137

Point update `value[s] = x`, query sum of the subtree of `s`.

- Place `value[v]` at position `tin[v]` of an array
- Update → point update at `tin[s]`
- Query → range sum on `[tin[s], tout[s]]`
- Fenwick tree (Week 24 preview): O(log n) each

Path-to-root queries — CSES 1138

Query: sum of values on the path from `s` up to the root.

- Changing `value[v]` by `d` changes the answer for **every node in v's subtree**
- → **range add** `d` on `[tin[v], tout[v]]`, **point query** at `tin[s]`
- Same Fenwick, used as a difference array
- Path `a – b` sum = `f(a) + f(b) − 2·f(lca) + value[lca]`

Alternative: LCA via RMQ

Full Euler tour (write a node every time you pass through it, 2n−1 entries):

```
tour : 1 2 1 3 4 3 5 3 1
depth: 0 1 0 1 2 1 2 1 0
```

- LCA(a, b) = node of **minimum depth** between first occurrences of a and b
- With a **sparse table** (Week 24): O(n log n) build, **O(1)** per query
- Use when q is huge; binary lifting is simpler and more flexible

Complexity summary

Technique	Preprocess	Query	Memory
Naive climb	O(n)	O(n)	O(n)
Binary lifting	O(n log n)	O(log n)	O(n log n)
Euler tour + sparse table	O(n log n)	O(1)	O(n log n)
Euler tour + Fenwick (subtree sums)	O(n)	O(log n)	O(n)

Common pitfalls

- `LOG` too small: need 2^LOG > max depth (use 18 for 2·10^5, 20 for 10^6)
- Filling `up[j]` before `up[j-1]` is complete for all nodes
- Root's parent left as garbage → jumps go out of bounds (use root itself or sentinel 0)
- Recursive DFS on a deep tree → **stack overflow**
- Forgetting phase 1 (equalizing depths) or returning `a` instead of `up[0][a]`
- Euler tour: `tout` must be the **last** tin in the subtree, not a separate counter

Practice problems

- CSES 1687 *Company Queries I* (k-th ancestor)
- CSES 1688 *Company Queries II* (LCA)
- CSES 1135 *Distance Queries*
- CSES 1137 *Subtree Queries* · CSES 1138 *Path Queries*
- CSES 1136 *Counting Paths* (LCA + difference on tree)
- Codeforces 1328E *Tree Queries*
- Codeforces 191C *Fools and Roads*
- Codeforces 609E *Minimum spanning tree for each edge*
- Codeforces 620E *New Year Tree* (Euler tour + segment tree)

Summary

- **Binary lifting**: `up[j][v] = up[j-1][up[j-1][v]]`, jumps by powers of two
- **LCA**: equalize depths, then lift both while ancestors differ
- `dist = depth[a] + depth[b] − 2·depth[lca]`
- **Euler tour**: subtree ↔ contiguous range `[tin, tout]`
- Tree problems become **array range problems** → next week: segment trees